Neuron-Level Knowledge Attribution in Large Language Models

Zeping Yu **Sophia Ananiadou**
Department of Computer Science, National Centre for Text Mining
The University of Manchester
{zeping.yu@postgrad. sophia.ananiadou@}manchester.ac.uk

arXiv:2312.12141v4 [cs.CL] 24 Sep 2024

Abstract

Identifying important neurons for final predictions is essential for understanding the mechanisms of large language models. Due to computational constraints, current attribution techniques struggle to operate at neuron level. In this paper, we propose a static method for pinpointing significant neurons. Compared to seven other methods, our approach demonstrates superior performance across three metrics. Additionally, since most static methods typically only identify "value neurons" directly contributing to the final prediction, we propose a method for identifying "query neurons" which activate these "value neurons". Finally, we apply our methods to analyze six types of knowledge across both attention and feed-forward network (FFN) layers. Our method and analysis are helpful for understanding the mechanisms of knowledge storage and set the stage for future research in knowledge editing. The code is available on https://github.com/zepingyu0512/neuron-attribution.

1 Introduction

Transformer-based large language models (LLMs) (Brown et al., 2020; Ouyang et al., 2022; Chowdhery et al., 2023) possess remarkable capabilities for storing factual knowledge, which is important for downstream tasks including question answering (Jiang et al., 2021) and reasoning (Rajani et al., 2019). While recent studies (Dai et al., 2021; Meng et al., 2022; Geva et al., 2023; Yu et al., 2023; Chen et al., 2024) have made significant progress in understanding knowledge localization and the information flow from inputs to predictions, it is still hard to identify exact parameters for knowledge storage in LLMs due to several reasons. Firstly, existing studies often depend on causal tracing (Pearl, 2001; Vig et al., 2020) and integrated gradients (Sundararajan et al., 2017) for knowledge attribution. However, many studies (Stolfo et al., 2023; Zhao et al., 2024; Wu et al., 2024a) point out that the computational complexity of forward and backward operations in these methods restricts their applicability to millions of neurons in LLMs, which are proved as fundamental units for storing knowledge (Geva et al., 2020; Dai et al., 2021; Geva et al., 2022; Nanda et al., 2023b). Secondly, while a few studies (Dar et al., 2022; Geva et al., 2022) have devised methods for analyzing neurons, they often lack comparisons with other methods. Therefore, how to identify important neurons in LLMs is still unclear. Lastly, existing methods typically concentrate on either attention or feed-forward network (FFN) module, often lacking evaluation of the other module. It is crucial to quantitatively compare the importance of both attention and FFN layers.

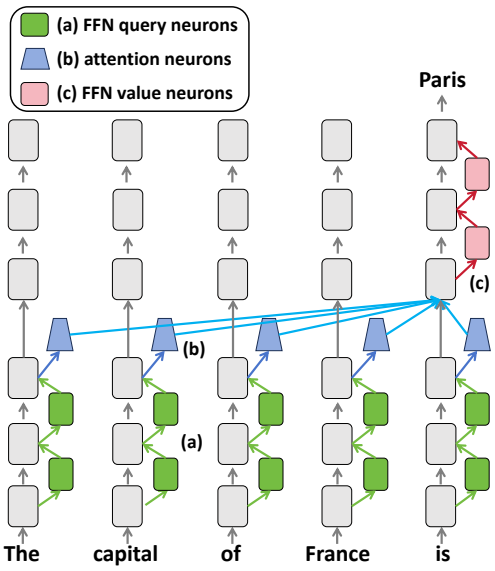

Figure 1: (a) Query neurons in shallow FFN layers. (b) Attention query/value neurons in attention heads. (c) Value neurons in deep FFN layers.

In this paper, we focus on neuron-level attribution methods. We analyze the distribution change caused by each neuron and discover that both the neuron's coefficient score and the final prediction's ranking, when projecting this neuron's subvalue into vocabulary space, play significant roles. Based

on this finding, we employ log probability increase as importance score, enabling the identification of neurons that contribute significantly to final predictions. Compared with seven other static methods, our proposed method achieves the best performance on three metrics. Furthermore, since the identified neurons directly contribute to the final predictions' probability, we also develop a static method to identify "query neurons" that aid in activating these "value neurons". Specifically, we calculate the inner products between the query neurons and value neurons as importance scores.

Based on our proposed methods, we analyze six types of knowledge in both attention and FFN layers, yielding numerous valuable insights (Figure 1): 1) Both attention and FFN layers can store knowledge, and all important neurons directly contribute to knowledge prediction are in deep layers. 2) In attention layers, knowledge with similar semantics (e.g. language, country, city) tends to be stored in the same heads. Knowledge with distinct semantics (e.g. country, color) is stored in different heads. 3) While numerous neurons contribute to the final prediction, intervening on a few value neurons (300) or query neurons (1000) can significantly influence the final prediction. 4) FFN value neurons are mainly activated by medium-deep attention value neurons, while these attention neurons are mainly activated by shallow/medium FFN query neurons.

Overall, our contributions are as follows:

a) We design a static method for neuron-level knowledge attribution in large language models. Compared with seven static methods, our method achieves the best performance under three metrics.

b) As the identified neurons usually directly contribute to the final predictions, we design a static method to identify the "query neurons" activating these "value neurons".

c) We analyze the localization of six types of knowledge in both attention and FFN layers. Our analysis is helpful for understanding the mechanisms of knowledge storage in language models.

2 Related Work

2.1 Attribution Methods for Transformers

Determining how to attribute the important parameters for final predictions is a crucial question. Gradient-based methods (Sundararajan et al., 2017; Kindermans et al., 2019; Miglani et al., 2020; Lundstrom et al., 2022) and causal tracing methods (Pearl, 2001; Vig et al., 2020; Meng et al., 2022; Goldowsky-Dill et al., 2023; Zhang and Nanda, 2023; Wu et al., 2024b; Hase et al., 2024) are widely utilized for this purpose. The core idea is calculating how much an internal module affects the final predictions, requiring multiple forward and/or backward operations (Wu et al., 2024a). Due to the computational overhead, these methods are usually applied on hidden states (Meng et al., 2022; Geva et al., 2023; Stolfo et al., 2023), rather than neurons. Another type of studies tend to require only one forward pass for each sentence, typically relying on saliency scores such as attention weights (Vig, 2019; Jaunet et al., 2021; Yeh et al., 2023; Wang et al., 2023; Li et al., 2023) and FFN neurons' coefficient scores (Geva et al., 2022; Lee et al., 2024). However, the validity of attributions is challenged by many studies (Serrano and Smith, 2019; Jain and Wallace, 2019; Wiegreffe and Pinter, 2019; Mohankumar et al., 2020; Ethayarajh and Jurafsky, 2021; Bai et al., 2021). Lack of evaluation methods results in an ongoing debate about the faithfulness of saliency score methods.

2.2 Mechanistic Interpretability

Mechanistic interpretility (Olah, 2022; Nanda et al., 2023a) aims to reverse engineer the circuits from inputs to the final prediction. An essential technology involves projecting internal vectors into the vocabulary space, where numerous studies have discovered interpretable results (Nostalgebraist, 2020; Geva et al., 2020, 2022; Dar et al., 2022; Pal et al., 2023). Most studies focus on analyzing the attention heads' roles in different cases and tasks (Elhage et al., 2021; Olsson et al., 2022; Wang et al., 2022; Hanna et al., 2023; Lieberum et al., 2023; Conmy et al., 2023; Gould et al., 2023). Also, superposition (Elhage et al., 2022; Nanda et al., 2023b; Gurnee et al., 2023) and dictionary learning (Bricken et al., 2023; He et al., 2024) are important for understanding neurons in transformers.

3 Methodology

In this section, we aim to locate important neurons for specific predictions. We introduce the background in Section 3.1, and analyze the distribution change caused by neurons in Section 3.2. Based on the analysis, we introduce our proposed method for locating the "value neurons" that contribute to the final predictions directly in Section 3.3, and propose a static method to locate the "query neurons" that activate these "value neurons" in Section 3.4.

3.1 Background

First, we introduce the inference pass from inputs to the final prediction. Given an input sentence $X = [t_1, t_2, ..., t_T]$ with T tokens, the model generates the next token's probability distribution y over B tokens in vocabulary V. The embedding matrix $E \in \mathbb{R}^{B \times d}$ transforms each t_i at position i into a word embedding $h_i^0 \in \mathbb{R}^d$. Then the word embeddings are transformed by $L+1$ transformer layers ($0th - Lth$), each has a multi-head self-attention layer (MHSA) and a FFN layer. The layer output h_i^l (position i, layer l) is the sum of the layer input h_i^{l-1} (previous layer's output), the attention output A_i^l, and the FFN output F_i^l:

$$h_i^l = h_i^{l-1} + A_i^l + F_i^l \tag{1}$$

Finally, the last position's Lth layer output is used to compute the final probability distribution y by multiplying the unembedded matrix $E_u \in \mathbb{R}^{B \times d}$:

$$y = softmax(E_u \, h_T^L) \tag{2}$$

Specifically, the attention layer's output is computed by a weighted sum over H heads on T positions, and the FFN layer's output is computed by a nonlinear function σ on two linear transformations.

$$A_i^l = \sum_{j=1}^{H} ATTN_j^l(h_1^{l-1}, h_2^{l-1} ..., h_T^{l-1}) \tag{3}$$

$$F_i^l = W_{fc2}^l \sigma(W_{fc1}^l (h_i^{l-1} + A_i^l)) \tag{4}$$

where $W_{fc1}^l \in \mathbb{R}^{N \times d}$ and $W_{fc2}^l \in \mathbb{R}^{d \times N}$ are two matrices. Geva et al. (2020) find FFN output can be represented as a weighted sum of FFN neurons:

$$F_i^l = \sum_{k=1}^{N} m_{i,k}^l fc2_k^l \tag{5}$$

$$m_{i,k}^l = \sigma(fc1_k^l \cdot (h_i^{l-1} + A_i^l)) \tag{6}$$

The FFN output F_i^l is computed by a weighted sum of $fc2$ vectors. $fc2_k^l$ is the kth column of W_{fc2}^l (named FFN subvalue), and its coefficient score $m_{i,k}^l$ is computed by non-linear σ on the inner product between the residual output $h_i^{l-1} + A_i^l$ and $fc1_k^l$ (named FFN subkey), the kth row of W_{fc1}^l. Similarly, the attention output A_i^l can be represented as a sum of head outputs, each being a weighted sum of value-output vectors on all positions:

$$A_i^l = \sum_{j=1}^{H} \sum_{p=1}^{T} \alpha_{i,j,p}^l W_{j,l}^o (W_{j,l}^v h_p^{l-1}) \tag{7}$$

$$\alpha_{i,j,p}^l = softmax(W_{j,l}^q h_i^{l-1} \cdot W_{j,l}^k h_p^{l-1}) \tag{8}$$

where $W_{j,l}^q, W_{j,l}^k, W_{j,l}^v, W_{j,l}^o \in \mathbb{R}^{d \times d/H}$ are the query, key, value and output matrices of the jth head in the lth layer. The query and key matrices compute the attention weight $\alpha_{i,j,p}^l$ on the pth position, then calculate the softmax function across all positions. The value and output matrices transform the pth position input vector into the pth value-output vector. Each head output is the weighted sum of value-output vectors on all positions.

Definition of "neuron". As discussed in Eq.5-6, the kth FFN neuron is the kth subvalue $fc2_k^l$, whose coefficient score $m_{i,k}^l$ is calculated by its corresponding subkey $fc1_k^l$. In Eq.7, each attention output can be represented as a direct addition of $T \times H$ vectors, when taking the position value-output vector $W_{j,l}^o(W_{j,l}^v h_p^{l-1})$ as fundamental units. Moreover, the position value-output vector can also be regarded as a weighted sum of attention neurons. Similar to the definition of FFN neurons, we regard the kth column of $W_{j,l}^o$ in Eq.7 as the kth attention subvalue in this head, whose subkey is the kth row of $W_{j,l}^v$. When taking the attention neurons as fundamental units, the final output is the sum of $L \times (T \times H \times d/H + N) + 1$ vectors.

3.2 Distribution Change Caused by Neurons

The final vector h_T^L has important information for predicting the final token. As it is computed by a direct sum of various neuron-level vectors, the relevant information for making the final prediction must be stored in one or many neurons. The final vector h_T^l can be regarded as the sum of one neuron v and another vector $x = h_T^l - v$. We consider the probability change $p(w|x+v) - p(w|x)$ caused by v for prediction token w. We aim to explore which components of v are significant for amplifying the probability change. This allows us to develop static methods for locating crucial neurons.

As the probability change is nonlinear, analyzing the exact contribution of neuron v is challenging. For a more concise analysis, we term the score $e_w \cdot x$ vector x's bs-value (before-softmax value) on token w, where e_w is the wth row of the unembedded matrix E_u. A token's bs-value directly corresponds to the probability of this token. Bs-values of all vocabulary tokens on vector x are:

$$bs(x) = [bs_1^x, bs_2^x, ..., bs_w^x, ..., bs_B^x] \tag{9}$$

For vector x, if bs_w^x is the largest among all the bs-values, the probability of word w will also be